MediCare

Hospital Appointment and Patient Management System

Project Detail	Description
Project Type	Django Full Stack Web Application
Domain	Healthcare / Hospital Management
Users	Patients, Doctors, Administrators
Backend	Python, Django, Django ORM
Database	SQLite for development; PostgreSQL or MySQL for deployment

PROJECT DOCUMENTATION

1. Project Overview

MediCare is a web-based Hospital Appointment and Patient Management System developed using Python and Django. Patients can browse doctor information and request appointments. Doctors can review requests and manage consultation schedules, while administrators maintain doctor profiles, departments, users, and appointment records. The system aims to simplify appointment coordination and reduce manual record handling.

2. Problem Statement

Traditional appointment booking often depends on phone calls, paper registers, or in-person visits. These processes can make it difficult to track appointment status, avoid scheduling conflicts, and maintain organized records. MediCare provides a centralized digital workflow with role-based access.

3. Project Objectives

- Allow patients to register, log in, browse doctors, and request appointments online.
- Maintain structured patient, doctor, department, and appointment records.
- Enable doctors or authorized staff to review and update appointment requests.
- Provide administrators with centralized management features.
- Improve appointment tracking and reduce manual coordination.
- Use authentication, authorization, and server-side validation.

4. User Roles and Responsibilities

Patient

- Register and log in securely.
- View doctor profiles and specializations.
- Request an appointment by selecting a doctor and preferred date/time.
- View appointment history and current status.
- Update permitted profile information.

Doctor

- Access a doctor dashboard.
- View assigned appointment requests.
- Review relevant patient and appointment details.
- Accept, reject, or update appointment status according to the workflow.
- Review upcoming appointments.

Administrator

- Manage patient and doctor accounts.
- Create and update doctor profiles and departments.
- View and manage appointment records.
- Oversee application data and workflows.

5. Main Application Modules

Module	Description
Authentication	Registration, login, logout, password handling, and role-based access.

Module	Description
Patient Management	Patient profiles and personal appointment history.
Doctor Management	Doctor profiles, qualifications, specialization, department, and availability.
Department Management	Create and maintain hospital departments or specializations.
Appointment Management	Appointment requests, review, status updates, and history.
Admin Dashboard	Centralized management of users, doctors, departments, and appointments.

6. Functional Requirements

- The system shall allow patients to register with valid details.
- The system shall authenticate users before granting access to protected pages.
- The system shall display doctor and department information.
- The system shall allow patients to submit appointment requests.
- The system shall store appointment date, time, patient, doctor, reason, and status.
- The system shall allow authorized users to update appointment status.
- The system shall restrict actions according to the user's role.
- Administrators shall be able to manage doctor and department records.

7. Non-Functional Requirements

- Security: Django authentication, permissions, CSRF protection, and input validation.
- Usability: clear navigation and understandable appointment status indicators.
- Reliability: graceful handling of invalid input and scheduling conflicts.
- Maintainability: reusable Django apps, models, views, forms, and templates.
- Scalability: production database and deployment configuration when needed.
- Performance: efficient queries and pagination for long lists.

8. Suggested Database Design

The following is a suggested relational design. Adjust field names and relationships to match the actual source code.

Entity	Suggested fields and relationships
User	id, username, email, password, role, is_active
PatientProfile	id, user_id (OneToOne), phone, date_of_birth, address
Department	id, name, description
DoctorProfile	id, user_id (OneToOne), department_id (ForeignKey), qualification, specialization, consultation_fee
Appointment	id, patient_id (ForeignKey), doctor_id (ForeignKey), appointment_date, appointment_time, reason, status, created_at

Relationships: one department may contain many doctors; one patient may create many appointments; one doctor may receive many appointment requests. A user account may be linked to a patient or doctor profile.

9. Appointment Workflow

- Patient registers or logs in.

- Patient browses doctor profiles and selects a doctor.
- Patient submits an appointment request with preferred date, time, and reason.
- The application validates and stores the request with an initial status such as Pending.
- The doctor or authorized staff reviews the request.
- The request is accepted, rejected, or updated according to the workflow.
- The patient checks the latest status from the dashboard.

10. Technology Stack

Technology	Purpose
Python	Core programming language.
Django	Backend framework, URL routing, ORM, authentication, and admin interface.
HTML5 / CSS3	Web page structure and styling.
Bootstrap	Responsive interface components, if used.
JavaScript	Client-side interactions and form enhancements, if required.
SQLite / PostgreSQL / MySQL	Relational database storage.
Git	Source code version control.

11. Suggested Django Project Structure

```
medicare_project/
  manage.py
  medicare_project/ (settings.py, urls.py, asgi.py, wsgi.py)
  accounts/ (models.py, views.py, urls.py, forms.py)
  patients/ (models.py, views.py, urls.py)
  doctors/ (models.py, views.py, urls.py)
  appointments/ (models.py, views.py, urls.py)
  templates/
  static/
```

Separating responsibilities into Django apps makes the code easier to understand, test, and maintain.

12. Validation and Security

- Validate required fields and ensure appointment dates are valid.
- Check that the selected doctor exists and is active.
- Prevent patients from accessing another patient's private records.
- Restrict doctors to appointments assigned to them.
- Use Django CSRF protection for form submissions.
- Use Django password hashing; never store plain-text passwords.
- Avoid exposing sensitive patient details in public pages, URLs, or logs.
- For real healthcare use, define suitable privacy, access, backup, and compliance procedures.

13. Testing

- Patient registration with valid and invalid data.
- Login and logout for each user role.

- Access control for patient, doctor, and administrator pages.
- Appointment creation with valid and invalid dates.
- Appointment status updates by authorized users.
- Prevention of unauthorized access to another patient's records.
- Doctor and department management by administrators.
- Handling duplicate or conflicting appointment slots, if implemented.

14. Installation and Execution

Typical local setup commands (install all additional dependencies from the project's requirements file):

```
python -m venv venv
# Windows: venv\\Scripts\\activate
# macOS/Linux: source venv/bin/activate
pip install django
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Open the local development address displayed by Django. For production, configure environment variables, a production database, static files, HTTPS, and a production WSGI/ASGI server.

15. Future Enhancements

- Email or SMS appointment confirmations and reminders.
- Doctor availability calendar and automated time-slot management.
- Appointment rescheduling and cancellation.
- Prescription and consultation-history management with access controls.
- Online payment integration for consultation fees.
- REST API support for mobile apps or external integrations.
- Dashboard analytics for appointment volume and department activity.

16. Conclusion

MediCare demonstrates how Django can support a structured web application for hospital appointment coordination and patient information management. By organizing patient, doctor, department, and appointment workflows in one system, it provides a foundation for future scheduling automation, notifications, payments, and additional healthcare administration features.

Documentation note: This document describes a suggested project design and typical Django implementation. Update database fields, modules, workflows, and technology details to reflect the exact features in your source code.